

High-Throughput LLM Serving on Linux with vLLM, CUDA, and Docker

 zihad.com.bd/posts/high-throughput-llm-serving-vllm-cuda-linux

September 21, 2026



Deploy and optimize production-grade Large Language Model (LLM) serving on Linux. Learn how PagedAttention eliminates KV cache waste, configure multi-GPU tensor parallelism, deploy vLLM with Docker, and achieve maximum tokens/sec with DeepSeek and Llama 3.

Serving open-source Large Language Models (such as Llama 3, DeepSeek, Mistral, and Qwen) in production using standard HuggingFace Transformers pipelines or naïve Flask/FastAPI wrappers quickly results in out-of-memory (OOM) crashes and abysmal request throughput.

The primary bottleneck in generative LLM inference is not raw compute—it is **GPU memory bandwidth** and **KV (Key-Value) cache fragmentation**.

vLLM is an open-source inference engine developed at UC Berkeley that utilizes **PagedAttention** (inspired by virtual memory paging in operating systems) to achieve **10x to 24x higher throughput** than standard serving runtimes.

Here is how to set up, configure, and tune an enterprise-ready vLLM server on Linux.

1. The PagedAttention Advantage

During autoregressive generation, an LLM must store the Key and Value vectors for all previous tokens in GPU VRAM (the KV cache).

Traditional serving frameworks allocate contiguous memory blocks based on the maximum possible request length (e.g., 8,192 tokens). Because actual user prompts and completions vary widely, **60% to 80% of GPU memory is locked up in unused, fragmented allocations**, preventing concurrent requests from processing.

PagedAttention fragments the KV cache into fixed-size virtual blocks stored in non-contiguous physical GPU memory pages. This eliminates internal fragmentation and enables dynamic memory sharing for complex parallel requests (such as parallel sampling and beam search).

2. Host Prerequisites: NVIDIA Drivers and Container Toolkit

Ensure your Linux host has the NVIDIA proprietary display drivers and the **NVIDIA Container Toolkit** installed so Docker containers can communicate directly with your GPUs.

Verify NVIDIA driver status:

```
nvidia
-smi
```

Install the NVIDIA Container Toolkit on Debian/Ubuntu:

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o
/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-
toolkit.list | \
  sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-
keyring.gpg] https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt update
sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctl runtime configure --runtime=docker
sudo systemctl restart docker
```

3. Deploy vLLM with Docker and GPU Acceleration

The cleanest way to serve models in production without managing complex local Python CUDA environments is using official vLLM container images.

Run the vLLM OpenAI-compatible server:

```
docker run -d \
  --name vllm-server \
  --runtime nvidia \
  --gpus all \
  -v
~/cache/huggingface:/root/.cache/huggingface \
-p 8000:8000 \
--ipc=host \
vllm/vllm-openai:latest \
--model meta-llama/Meta-Llama-3-8B-Instruct \
--gpu-memory-utilization 0.90 \
--max-model-len 8192
```

Essential Flags Explained:

- `--runtime nvidia --gpus all`: Passes host GPUs directly into the container.
- `--ipc=host`: Uses the host's shared memory space, preventing PyTorch inter-process communication failures.
- `--gpu-memory-utilization 0.90`: Dedicates 90% of total GPU VRAM to model weights and the dynamic KV cache pool.
- `--max-model-len 8192`: Caps context window to prevent memory exhaustion on extreme prompts.

4. Multi-GPU Serving with Tensor Parallelism

When serving models that exceed the VRAM of a single graphics card (e.g., Llama-3-70B, DeepSeek-V2), or when you want to minimize latency on large models, distribute computation across multiple GPUs using **Tensor Parallelism**:

```
docker run -d \
  --name vllm-multi-gpu \
  --runtime nvidia \
  --gpus '"device=0,1,2,3"' \
  -v
~/cache/huggingface:/root/.cache/huggingface \
-p 8000:8000 \
--ipc=host \
vllm/vllm-openai:latest \
--model meta-llama/Meta-Llama-3-70B-Instruct \
--tensor-parallel-size 4 \
--gpu-memory-utilization 0.92
```

Note: The `--tensor-parallel-size` must divide evenly into the number of available GPUs (e.g., 2, 4, or 8).

5. Leverage Quantization (AWQ / GPTQ / FP8)

If you are running on consumer or mid-tier enterprise GPUs (such as RTX 4090, A5000, or L4), full 16-bit floating point (bfloat16) models may severely restrict your concurrent batch size.

Running **AWQ (Activation-aware Weight Quantization)** or **FP8** compressed checkpoints halves the VRAM needed for weights, leaving significantly more memory free for larger KV cache pools and higher concurrent user sessions:

```
# Serve an AWQ 4-bit quantized model
vllm serve casperhansen/llama-3-70b-instruct-
awq \
  --quantization awq \
  --dtype auto \
  --tensor-parallel-size 2 \
  --port 8000
```

6. Querying the OpenAI-Compatible API

vLLM provides a drop-in replacement API compatible with the standard OpenAI API specification. You can immediately point your existing applications, LangChain, or LiteLLM gateways to your local instance.

Test inference with curl:

```
curl http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta-llama/Meta-Llama-3-8B-Instruct",
    "messages": [
      {"role": "system", "content": "You are a high-performance Linux engineering
assistant."},
      {"role": "user", "content": "Explain how PagedAttention manages memory."}
    ],
    "temperature": 0.2,
    "max_tokens": 256
  }'
```

In Python:

```

from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:8000/v1",
    api_key="none" # Authentication can be enforced with --api-key
)

response = client.chat.completions.create(
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    messages=[{"role": "user", "content": "What are the advantages of vLLM?"}],
    stream=True
)

for chunk in response:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="", flush=True)

```

7. Production Monitoring and Healthchecks

vLLM exposes native **Prometheus metrics** on `/metrics`, allowing you to monitor GPU cache saturation and request queuing in Grafana:

- `vllm:num_requests_running`: Number of currently active generation requests.
- `vllm:num_requests_waiting`: Queued requests waiting for free KV cache pages.
- `vllm:gpu_cache_usage_factor`: Percentage of allocated KV cache in use.

If `vllm:gpu_cache_usage_factor` continuously hovers near 1.0 while `num_requests_waiting` spikes, either add more GPUs with tensor parallelism, downscale `--max-model-len`, or implement request rate-limiting at your reverse proxy layer.